

Lab Practice – II

M.Tech CSE | Semester II (2025–26)

1. Course Overview

Course Description

This laboratory course provides hands-on exposure to advanced deep learning models, data analytics, visualization tools, and end-to-end MLOps practices. Students implement real-world problems involving image processing, NLP, recommender systems, healthcare AI, and cloud-deployed machine learning pipelines aligned with industry and research expectations.

2. Course Objectives

- Implement advanced deep learning models for vision, language, signals, and sensor data
- Apply data preprocessing, modeling, and visualization techniques on real datasets
- Design and deploy scalable ML systems using MLOps best practices
- Build reproducible and automated ML pipelines using industry-standard tools

3. Course Outcomes (CO)

- **CO1:** Build end-to-end AI-based systems for real-world problem domains
- **CO2:** Apply deep learning techniques to diverse datasets (images, text, signals, sensors)
- **CO3:** Create analytical dashboards for data interpretation and decision support
- **CO4:** Deploy and manage machine learning models using MLOps best practices

4. Software & Hardware Requirements

Hardware - Minimum 8 GB RAM (16 GB recommended) - GPU support (NVIDIA CUDA-enabled – optional but recommended)

Software - OS: Linux / Windows / macOS - Python 3.9+ - Anaconda / Miniconda - Frameworks: TensorFlow / PyTorch - Visualization: Matplotlib, Seaborn, Plotly, Power BI / Tableau - MLOps Tools: Git, DVC, MLflow, Docker, FastAPI - IDE: VS Code / Jupyter Notebook

5. Lab Guidelines

- Students must **select any FOUR problem statements** for implementation
- Each problem must be treated as a **mini-project**
- Proper code versioning and experiment tracking is mandatory
- Submissions must include code, report, results, and demo

6. Standard Experiment Template (To be followed for all problems)

Aim

To design and implement an intelligent system using Deep Learning, Data Modeling & Visualization (DMV), and MLOps techniques.

Problem Statement

(Brief description of the selected problem)

Dataset Description

- Source
- Features
- Size and format

Tools & Technologies

- DL Framework
- Visualization tools
- MLOps components

Methodology

1. Data acquisition and preprocessing
2. Model architecture design
3. Training and hyperparameter tuning
4. Evaluation and performance analysis
5. Visualization and dashboard creation
6. Model packaging and deployment

Evaluation Metrics

- Accuracy / Precision / Recall / F1-score
- Loss curves
- Domain-specific metrics

Outcome

Summary of results and observations

7. List of Experiments / Problem Statements

Experiment 1: Intelligent Image Classification and Visual Analytics Platform

Aim:

To build an image classification system using deep learning and visualize model performance through analytical dashboards with an automated MLOps pipeline.

Tasks: - Implement CNN/Deep MLP for image classification - Plot accuracy, loss, confusion matrix - Track experiments using MLflow - Dockerize and deploy the model

CO Mapping: CO1, CO2, CO3, CO4

Experiment 2: Smart Plant Health Monitoring System

Aim:

To detect plant diseases from leaf images and present disease trends through interactive dashboards.

Tasks: - CNN-based multi-class disease classification - EDA on plant disease dataset - Tableau / Power BI dashboard - Git + DVC based data versioning - REST API deployment

CO Mapping: CO1, CO2, CO3, CO4

Experiment 3: Sentiment Intelligence System for Product Reviews

Aim:

To classify customer sentiment using LSTM-based deep learning models and deploy the system as an inference service.

Tasks: - Text preprocessing and tokenization - LSTM sentiment classification - Sentiment distribution visualization - MLflow experiment tracking - API-based deployment

CO Mapping: CO1, CO2, CO3, CO4

Experiment 4: Smart Recommendation and Insight Dashboard System

Aim:

To design a personalized recommendation engine and analyze user behavior using dashboards.

Tasks: - Deep learning-based recommendation model - User-item interaction analysis - BI dashboards for insights - Automated training and deployment pipeline

CO Mapping: CO1, CO2, CO3, CO4

Experiment 5: Smart Rainfall Prediction and Decision Support System

Aim:

To predict rainfall using historical weather data and present spatio-temporal analytics.

Tasks: - Train DL/ML rainfall prediction model - Time-series and geospatial visualization - Git + DVC + MLflow integration - CI/CD-enabled deployment

CO Mapping: CO1, CO2, CO3, CO4

Experiment 6: AI-Driven Parkinson's Rehabilitation Monitoring System

Aim:

To monitor rehabilitation progress of Parkinson's patients using sensor/video data.

Tasks: - CNN/LSTM/pose-estimation model - Symptom severity analysis - Rehabilitation dashboards - Remote monitoring pipeline

CO Mapping: CO1, CO2, CO3, CO4

Experiment 7: Parkinson's Disease Detection Using Brain Imaging

Aim:

To detect Parkinson's disease from MRI/DaTscan images using deep learning.

Tasks: - Image preprocessing and augmentation - CNN / transfer learning model - Brain slice visualization - Dockerized inference service

CO Mapping: CO1, CO2, CO3, CO4

Experiment 8: EEG-Based Epileptic Seizure Detection System

Aim:

To detect epileptic seizures from EEG signals using deep learning.

Tasks: - EEG signal preprocessing - CNN/LSTM-based classification - Time-series visualization - Reproducible training pipeline

CO Mapping: CO1, CO2, CO3, CO4

8. Assessment Scheme (Continuous Assessment – 50 Marks)

Component Marks

Problem	Results
Understanding (DL + DMV + MLOps)	6
Algorithm Implementation &	14

Mini Projects (3 Implementations) 25

Attendance & Participation 5

Total 50

9. Documentation & Submission Guidelines

- Structured PDF report
- GitHub repository link
- Screenshots of dashboards
- MLflow experiment logs
- Dockerfile & API demo

10. Expected Learning Outcomes

By completing this laboratory, students will gain industry-ready skills in deep learning model development, analytics, visualization, and MLOps-driven deployment pipelines suitable for research and production environments.

Prepared By

Dr. Reena Pagare

Experiment 1: Intelligent Image Classification and Visual Analytics Platform

Aim:

To build an image classification system using deep learning and visualize model performance through analytical dashboards with an automated MLOps pipeline.

Tasks: - Implement CNN/Deep MLP for image classification - Plot accuracy, loss, confusion matrix - Track experiments using MLflow - Dockerize and deploy the model

Intelligent Image Classification & Visual Analytics Platform

1 Dataset & Problem Setup

Recommended datasets

- **MNIST (quick validation)**
 CIFAR-10 (better for CNN)
- **Custom image dataset (advanced)**

```
from tensorflow.keras.datasets import cifar10
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

X_train, X_test = X_train / 255.0, X_test / 255.0
```

2 Model Implementation

Option A: Deep MLP (Baseline)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Dropout
```

```
mlp = Sequential([
    Flatten(input_shape=(32, 32, 3)),
    Dense(512, activation='relu'),
    Dropout(0.5),
    Dense(256, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
mlp.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Option B: CNN (Recommended)

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D
```

```
cnn = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
```

```
Dense(128, activation='relu'),  
Dense(10, activation='softmax')  
)
```

```
cnn.compile(optimizer='adam',  
            loss='sparse_categorical_crossentropy',  
            metrics=['accuracy'])
```

3 Training & Evaluation

```
history = cnn.fit(X_train, y_train,  
                  epochs=10,  
                  batch_size=64,  
                  validation_split=0.2)
```

```
test_loss, test_acc = cnn.evaluate(X_test, y_test)
```

4 Visual Analytics (Performance Plots)

Accuracy & Loss Curves

```
import matplotlib.pyplot as plt
```

```
plt.plot(history.history['accuracy'], label='Train Acc')  
plt.plot(history.history['val_accuracy'], label='Val Acc')  
plt.legend()  
plt.title("Accuracy Curve")  
plt.show()
```

```
plt.plot(history.history['loss'], label='Train Loss')  
plt.plot(history.history['val_loss'], label='Val Loss')  
plt.legend()  
plt.title("Loss Curve")  
plt.show()
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix  
import seaborn as sns
```

```
y_pred = cnn.predict(X_test).argmax(axis=1)  
cm = confusion_matrix(y_test, y_pred)
```

```
sns.heatmap(cm, annot=True, fmt='d')  
plt.title("Confusion Matrix")  
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
plt.show()
```

These plots can later be exported to Power BI / Tableau.

5 MLflow Experiment Tracking

```
pip install mlflow
mlflow ui
```

```
import mlflow
import mlflow.tensorflow
```

```
with mlflow.start_run():
    mlflow.log_param("epochs", 10)
    mlflow.log_param("batch_size", 64)
```

```
history = cnn.fit(X_train, y_train, epochs=10)
loss, acc = cnn.evaluate(X_test, y_test)
```

```
mlflow.log_metric("test_accuracy", acc)
mlflow.tensorflow.log_model(cnn, "cnn_model")
```

6 Save Model for Deployment

```
cnn.save("model/image_classifier.h5")
```

7 API Deployment (FastAPI)

```
from fastapi import FastAPI, UploadFile
import tensorflow as tf
import numpy as np
from PIL import Image
import io
app = FastAPI()
model = tf.keras.models.load_model("model/image_classifier.h5")
```

```
@app.post("/predict")
async def predict(file: UploadFile):
    img = Image.open(io.BytesIO(await file.read()))
    img = img.resize((32,32))
    img = np.array(img)/255.0
    img = img.reshape(1,32,32,3)

    pred = model.predict(img)
    return {"class": int(pred.argmax()), "confidence": float(pred.max())}
```


Run:

uvicorn app:app --reload

8 Dockerization

Dockerfile

FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

requirements.txt

tensorflow

fastapi

uvicorn

pillow

mlflow

Build & Run:

docker build -t image-classifier .

docker run -p 8000:8000 image-classifier

9 Automated MLOps Pipeline (Conceptual Flow)

Data Ingestion → Model Training → MLflow Tracking

↓

Model Registry → Docker Build → API Deployment

Can be extended with:

- **GitHub Actions (CI/CD)**
- **Kubernetes (scaling)**
- **Prometheus (monitoring)**

Final Project Structure

Image-Classification-MLOps/

```
|— data/  
|— notebooks/  
|— model/
```

```
|— app.py
|— Dockerfile
|— requirements.txt
|— mlruns/
|— README.md
```

Learning Outcomes

Students will:

- Build CNN-based image classifiers •

Analyze model performance visually •

Track experiments using MLflow •

Deploy models as APIs

- Apply container-based MLOps practices

Experiment 2: Smart Plant Health Monitoring System

Aim:

To detect plant diseases from leaf images and present disease trends through interactive dashboards.

Tasks: - CNN-based multi-class disease classification - EDA on plant disease dataset - Tableau / Power BI dashboard - Git + DVC based data versioning - REST API deployment

Dataset (recommended):

- PlantVillage dataset (Kaggle / GitHub)
- Structure:

PlantVillage/

```
|— Tomato__Early_blight/
|— Tomato__Late_blight/
|— Potato__Healthy/
|— ...
```

Goal:

Multi-class classification : *Leaf Image* → *Disease Type*

2. Exploratory Data Analysis (EDA)

Tools: Python, Pandas, Matplotlib, Seaborn

Steps

- Count images per class (class imbalance check)
- Sample visualization of leaves
- Image size distribution
- Healthy vs Diseased ratio

Example EDA snippets (conceptual)

- Bar chart of disease classes
- Random 5 images per disease
- Heatmap of class distribution

Outcome:

- Data understanding report
- Insight: which diseases dominate, imbalance issues

This EDA output later feeds directly into the dashboard.

3) CNN-Based Disease Classification

Framework: TensorFlow / Keras (preferred for students)

Model Options

- Baseline CNN (from scratch) – for learning
- Transfer Learning (recommended):
 - MobileNetV2 / ResNet50 / EfficientNet

Workflow

1. Image preprocessing:
 - Resize (224×224)
 - Normalization
2. Train/Validation split (70-20-10)
3. Data augmentation (rotation, flip, zoom)
4. Model training
5. Evaluation:
 - Accuracy
 - Confusion matrix
 - Precision / Recall / F1

Deliverables

- Trained model (.h5 or .pt)

- Training curves
- Classification report

4. Data & Model Versioning (Git + DVC)

This is where it becomes MLOps-ready

Setup

`git init`

`dvc init`

Track dataset

`dvc add data/PlantVillage`

`git add data/PlantVillage.dvc .gitignore`

`git commit -m "Add PlantVillage dataset"`

Track model artifacts

`dvc add models/plant_disease_model.h5`

`git commit -m "Add trained CNN model"`

Benefits

- Dataset version control
Reproducible experiments
- Easy rollback for students

5 Model Inference via REST API

Framework: FastAPI (lightweight & clean)

API Design

Endpoint: `POST /predict`

Input: Leaf image

Output:

```
{  
  "disease": "Tomato_Early_Blight",  
  "confidence": 0.94  
}
```

Steps

- Load trained CNN model
- Image preprocessing inside API
- Return prediction JSON

Test using:

- Swagger UI (`/docs`)
- Postman
- Simple frontend / curl

6 Data Logging for Dashboard

Every prediction should be logged

Store Prediction Data

Use:

- CSV
- SQLite / PostgreSQL

Fields:

- Timestamp
- Crop type
- Disease
- Confidence score

Example:

2026-02-01 | Tomato | Early Blight | 0.94

7 Dashboard (Tableau / Power BI)

This makes the experiment *feel real-world* ❖❖

Connect Dashboard to:

- CSV / Database containing predictions + EDA stats

Suggested Visuals

- ❖❖❖ Disease trend over time
- ❖❖❖ Disease distribution (pie)
- ❖❖❖ Crop-wise disease frequency
- ❖❖❖ Model confidence histogram

Outcome

Interactive Plant Health Monitoring Dashboard

8 Final Project Structure

Plant-Health-Monitoring/

```
├── data/ (DVC tracked)
├── notebooks/
│   ├── eda.ipynb
│   └── training.ipynb
├── models/
├── api/
│   └── app.py
├── dashboard/
│   └── plant_health.pbix / tableau.twb
├── dvc.yaml
└── README.md
```

9) Learning Outcomes (Perfect for Distributed/Applied ML)

Students learn:

- Deep Learning (CNNs)
- EDA & visualization
- MLOps basics (Git + DVC)
- Model deployment (REST API)
- Data-driven decision dashboards

Experiment 3: Sentiment Intelligence System for Product Reviews

1 Dataset Selection

Recommended datasets

- IMDb Movie Reviews (binary sentiment)
- Amazon Product Reviews (multi-domain)

Format

review_text | sentiment

"This phone is amazing" | positive

"Battery life is poor" | negative

2 Text Preprocessing & Tokenization

Tools: Python, NLTK / spaCy, TensorFlow Keras

Steps

1. Lowercasing
2. Remove HTML tags & punctuation
3. Remove stopwords
4. Tokenization & padding

Code (Preprocessing + Tokenizer)

```
import re
import nltk
from nltk.corpus import stopwords
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

def clean_text(text):
```

```

text = text.lower()
text = re.sub(r'<.*?>', '', text)
text = re.sub(r'[^a-z\s]', '', text)
text = ' '.join([w for w in text.split() if w not in stop_words])
return text

df['clean_review'] = df['review'].apply(clean_text)

tokenizer = Tokenizer(num_words=10000, oov_token="<OOV>")
tokenizer.fit_on_texts(df['clean_review'])

sequences = tokenizer.texts_to_sequences(df['clean_review'])
padded_seq = pad_sequences(sequences, maxlen=200, padding='post')

```

3.LSTM-Based Sentiment Classification

Framework: TensorFlow / Keras

Model Architecture

- Embedding Layer
- LSTM
- Dense (Sigmoid / Softmax)

Code (LSTM Model)

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

model = Sequential([
    Embedding(input_dim=10000, output_dim=128, input_length=200),
    LSTM(128, return_sequences=False),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    padded_seq, labels,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)

```

Evaluation

- Accuracy
- Confusion Matrix
- Precision / Recall / F1-score

4 Sentiment Distribution Visualization

Goal: Understand customer opinion trends

Code

```
import matplotlib.pyplot as plt
```

```
sentiments = df['sentiment'].value_counts()
```

```
plt.bar(sentiments.index, sentiments.values)
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.show()
```

5 MLflow Experiment Tracking (MLOps)

Tracks:

- Parameters
- Metrics
- Model artifacts

Setup

```
pip install mlflow
mlflow ui
```

Code (MLflow Logging)

```
import mlflow
import mlflow.tensorflow
```

```
with mlflow.start_run():
    mlflow.log_param("epochs", 5)
    mlflow.log_param("batch_size", 64)
```

```
model.fit(padded_seq, labels, epochs=5, batch_size=64)
```

```
loss, acc = model.evaluate(padded_seq, labels)
mlflow.log_metric("accuracy", acc)
```

```
mlflow.tensorflow.log_model(model, "sentiment_model")
```


Students visually compare runs via MLflow UI.

6 API-Based Deployment (Inference Service)

Framework: FastAPI

API Workflow

- Accept review text
- Preprocess text
- Predict sentiment
- Return JSON output

Code (FastAPI App)

```
from fastapi import FastAPI
```

```
import tensorflow as tf
```

```
import numpy as np
```

```
app = FastAPI()
```

```
model = tf.keras.models.load_model("models/sentiment_model")
```

```
tokenizer = pickle.load(open("tokenizer.pkl", "rb"))
```

```
@app.post("/predict")
```

```
def predict_sentiment(review: str):
```

```
    cleaned = clean_text(review)
```

```
    seq = tokenizer.texts_to_sequences([cleaned]) padded =  
    pad_sequences(seq, maxlen=200, padding='post')
```

```
    prediction = model.predict(padded)[0][0]
```

```
    sentiment = "Positive" if prediction > 0.5 else "Negative"
```

```
    return {
```

```
        "review": review,
```

```
        "sentiment": sentiment,
```

```
        "confidence": float(prediction)
```

```
    }
```

Run API

```
uvicorn app:app --reload
```

Test using:

- Swagger UI ([/docs](#))
- Postman

7. Final Project Structure

Sentiment-Intelligence-System/

```
├── data/
├── notebooks/
│   ├── eda.ipynb
│   └── lstm_training.ipynb
├── models/
│   └── sentiment_model
├── api/
│   └── app.py
├── mlruns/
└── README.md
```

8 Learning Outcomes (Strong for Curriculum)

Students gain:

- NLP preprocessing & tokenization
- LSTM-based deep learning
- Model evaluation & visualization
- MLflow experiment tracking
- Real-time AI inference using APIs

Experiment 3: Sentiment Intelligence System for Product Reviews

Aim:

To classify customer sentiment using LSTM-based deep learning models and deploy the system as an inference service.

Tasks: - Text preprocessing and tokenization - LSTM sentiment classification - Sentiment distribution visualization - MLflow experiment tracking - API-based deployment

Recommended datasets

- IMDb Movie Reviews (binary sentiment)
- Amazon Product Reviews (positive / negative)

Assumed format

```
review_text | sentiment
-----
"Very good battery life" | 1
"Poor quality product" | 0
```

2Text Preprocessing & Tokenization

Tools: Python, NLTK, TensorFlow (Keras)

Steps

- Convert text to lowercase
- Remove HTML tags, punctuation, numbers
- Remove stopwords
- Tokenize and pad sequences

Code

```
import re
import nltk
import pandas as pd
from nltk.corpus import stopwords
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'^a-z\s', '', text)
    text = ' '.join([w for w in text.split() if w not in
stop_words])
    return text

df = pd.read_csv("reviews.csv")
df['clean_review'] = df['review_text'].apply(clean_text)

tokenizer = Tokenizer(num_words=10000, oov_token="<OOV>")
tokenizer.fit_on_texts(df['clean_review'])
```

```
sequences = tokenizer.texts_to_sequences(df['clean_review'])
X = pad_sequences(sequences, maxlen=200, padding='post') y =
df['sentiment']
```

3LSTM-Based Sentiment Classification

Model Architecture

- Embedding Layer
- LSTM Layer
- Dense Output Layer

Code

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense,
Dropout

model = Sequential([
    Embedding(input_dim=10000, output_dim=128, input_length=200),
    LSTM(128),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X, y,
    epochs=5,
    batch_size=64,
    validation_split=0.2
```

)

4 Sentiment Distribution Visualization

Purpose: Visualize customer opinion trends

Code

```
import matplotlib.pyplot as plt

sentiment_counts = df['sentiment'].value_counts()
labels = ['Negative', 'Positive']

plt.bar(labels, sentiment_counts.values)
plt.xlabel("Sentiment")
plt.ylabel("Number of Reviews")
plt.title("Sentiment Distribution of Product Reviews")
plt.show()
```

This output can be exported to Tableau / Power BI if required.

5 MLflow Experiment Tracking

Tracks:

- Hyperparameters
- Accuracy
- Trained model

Setup

```
pip install mlflow
mlflow ui
```

MLflow Logging Code

```
import mlflow
import mlflow.tensorflow
```

```
with mlflow.start_run():
    mlflow.log_param("epochs", 5)
    mlflow.log_param("batch_size", 64)

    model.fit(X, y, epochs=5, batch_size=64, validation_split=0.2)

    loss, accuracy = model.evaluate(X, y)
    mlflow.log_metric("accuracy", accuracy)

    mlflow.tensorflow.log_model(model, "sentiment_lstm_model")
Access UI:
🔗🔗 http://localhost:5000
```

6 Save Model & Tokenizer

```
import pickle

model.save("sentiment_model.h5")
pickle.dump(tokenizer, open("tokenizer.pkl", "wb"))
```

7 API-Based Deployment (Inference Service)

Framework: FastAPI

API Functionality

- Accept review text
- Preprocess text
- Predict sentiment
- Return result as JSON

FastAPI Code

```
from fastapi import FastAPI
import tensorflow as tf
import pickle
import numpy as np
```

```

from tensorflow.keras.preprocessing.sequence import pad_sequences

app = FastAPI()

model = tf.keras.models.load_model("sentiment_model.h5")
tokenizer = pickle.load(open("tokenizer.pkl", "rb"))

def preprocess_text(text):
    text = clean_text(text)
    seq = tokenizer.texts_to_sequences([text])
    padded = pad_sequences(seq, maxlen=200, padding='post')
    return padded

@app.post("/predict")
def predict_sentiment(review: str):
    processed = preprocess_text(review)
    prediction = model.predict(processed)[0][0]    sentiment =

"Positive" if prediction > 0.5 else "Negative"

    return {
        "review": review,
        "sentiment": sentiment,
        "confidence": float(prediction)
    }

```

Run API

```
uvicorn app:app --reload
```

Test at:

<http://127.0.0.1:8000/docs>

8Final Project Structure

Sentiment-Intelligence-System/

```

├── data/
│   └── reviews.csv

```

```
|— notebooks/
|   |— sentiment_training.ipynb
|— models/
|   |— sentiment_model.h5
|   |— tokenizer.pkl
|— api/
|   |— app.py
|— mlruns/
|— README.md
```

Outcome of the Experiment

Students will be able to:

- Preprocess and tokenize textual data •
- Build LSTM-based sentiment classifiers •
- Visualize customer sentiment distribution •
- Track ML experiments using MLflow
- Deploy deep learning models as REST APIs